AnyGameRenderer: Entity-Consistent Generative Rendering Across Games and an Entity-Level Benchmark

Ruicong Liu*, Liangyang Ouyang*, Fengbo Lan, Zhixiang Wang, Kaipeng Zhang

Alaya Lab

AI coding agents can now build small playable games in minutes, but the results usually look like programmer art: gray blockouts, colored geometric primitives, and flat semantic blobs. We present AnyGameRenderer, a video-to-video generative renderer that turns such footage into high-quality visuals with characters, motion detail, and effects. Throughout, it keeps the layout, camera, and every entity's position and timing intact. Our AnyGameRenderer generalizes across a wide range of genres that agents build, such as tower defense, auto-battlers, top-down survivors, turn-based battles, kart racing, and crowd runners. AnyGameRenderer takes plain video as its game-state input, with no complex representations such as engine G-buffers, per-object class labels, or low-poly proxies. Gray blockouts, colored primitives, semantic maps, and wireframes all work as input. It also preserves entity correspondence: group counts stay correct, and entities appear and disappear on the right frame. We implement AnyGameRenderer by LoRA fine-tuning MiniMax-H3 ref2va, a 33B reference-video-conditioned video generation model. Training uses ∼10k paired clips from three sources: (i) multi-stream rendering data from 38 agent-made games exported from GEAR, an AI game-creation tool; (ii) two-stream rendering data from Unreal Engine, with programmatically controlled crowds, count changes, and entity scales; and (iii) real gameplay recordings, with pairs reconstructed via 3D-box detection. To evaluate the entity correspondence, we propose an entity-level benchmark of 59 held-out cases with 753 entities, scored against ground-truth boxes. Compared with the base model, AnyGameRenderer better preserves entity correspondence: temporal match rises from 50.9% to 62.9%, missed count changes fall from 34.7% to 21.3%, and fewer entities are hallucinated or left unrendered.

Project Page: https://alaya-lab.github.io/AnyGameRenderer
Code: https://github.com/AlayaLab/AnyGameRenderer
Correspondence: Kaipeng Zhang
Note: * Equal contribution
Date: October 22, 2026

ALAYA LAB

1 Introduction

Coding agents and tools, such as GEAR [1] game-creation system, can now turn a one-paragraph prompt into a playable game in minutes, complete with game logic, physics, camera, win conditions, and working HUD [7]. What they cannot produce is good art. The output usually contains gray boxes, colored cylinders, and cones on flat-shaded ground. Human studios spend months replacing this kind of "programmer art" through modeling, texturing, and lighting. Splitting a game into an engine that manages state and a generative model that renders pixels has quickly become a common idea. Concurrent systems such as CWM [8], PWM [15], Matrix-Game 4.0 [32], StreamRender-H3 [29], Marionette [26], and SolarRenderer [33] all adopt it in some form. We do not argue for this setting. Instead, we build a rendering layer that is more *general* in what it accepts and more *faithful* in what it keeps.

As shown in Figure 1, AnyGameRenderer takes a crude gameplay recording and a text prompt (optionally with a style anchor image), and outputs the same gameplay re-rendered in the prompted style, frame-aligned with the input. It adds characters, motion detail, and effects, while keeping the layout, camera, and

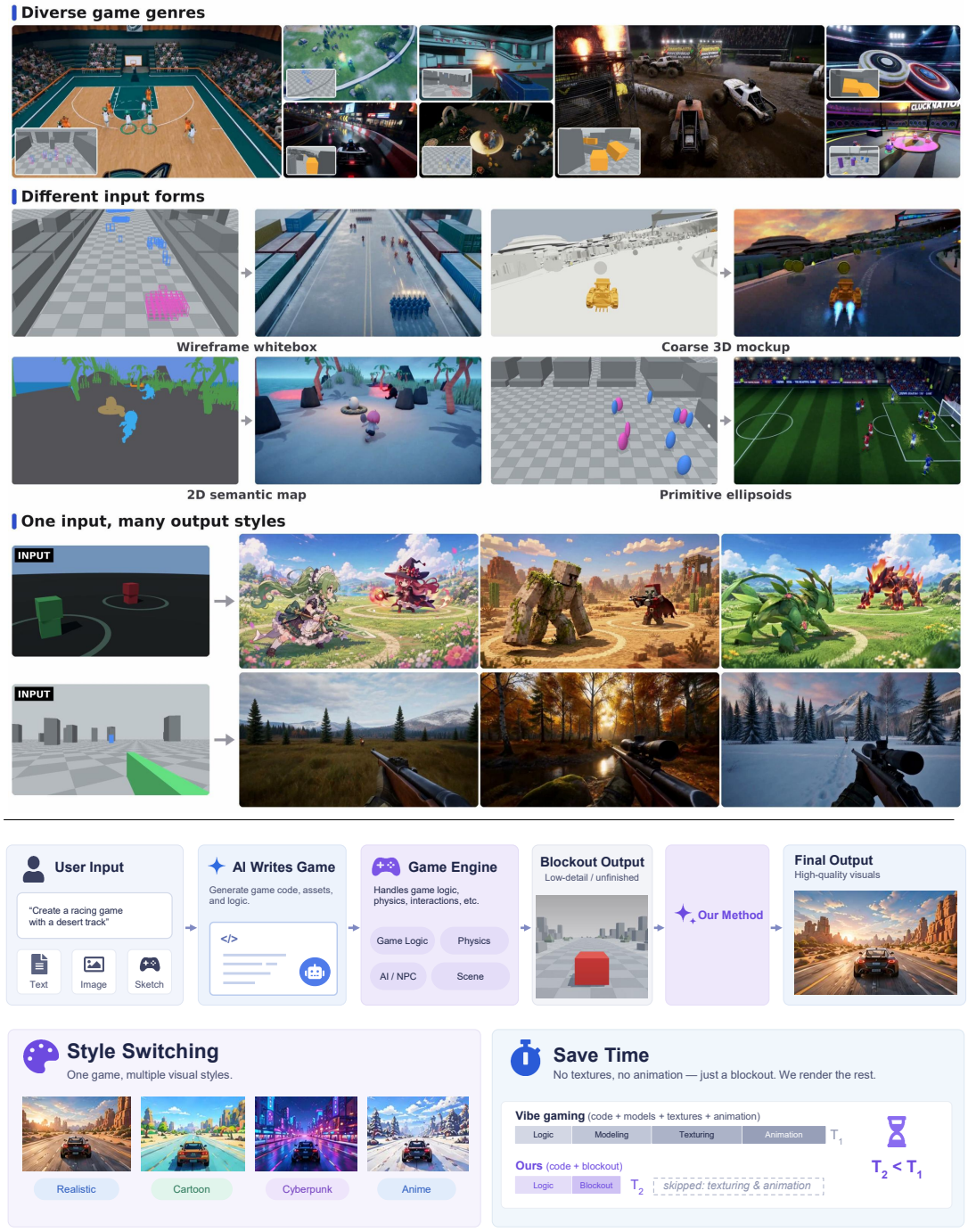


Figure 1 AnyGameRenderer turns crude gameplay into polished game visuals. *Top:* one checkpoint handles diverse genres and input types, and renders one input in many styles. Insets show the crude input. *Bottom:* an agent writes the game and the engine outputs a blockout, which AnyGameRenderer renders into the final visuals. This allows style switching and skips modeling, texturing and animation.

every entity's position and timing. This brings two practical benefits. A game can easily switch to a new visual style by changing only the prompt. It also makes vibe gaming faster, since a game only needs code and a blockout, without modeling, texturing or animation.

There are three properties that set AnyGameRenderer apart. *(1) It generalizes across genres.* Prior and concurrent works are demonstrated mainly on third-person, first-person, or driving footage, with limited

coverage of the genres that agents actually produce. As illustrated in Figure 1, AnyGameRenderer generalizes across a wide range of genres, including tower defense, auto-battlers, top-down survivors, turn-based battles, kart racing, crowd runners, arena survival, co-op games, and shooters. In addition, it accepts diverse input types, including gray blockouts, colored primitives, semantic maps, wireframes, and coarse mockup.

(2) It keeps entity correspondence. The hard part of game rendering is bookkeeping: twelve enemies become nine when three die; a reinforcement wave walks in at t=4.2 s; a tiny far-away mob must not dissolve into fog. AnyGameRenderer substantially improves entity correspondence: group counts stay correct, entities appear and disappear on the right frame, and fewer small entities are lost. The zero-shot base model often fails here: entities never appear, colored blocks stay unrendered, and characters appear even when they are not in the input.

(3) Its input is plain video. Many concurrent systems rely on costly or complex representations, such as per-object class labels (StreamRender-H3, CWM), state-augmented 3D boxes (PWM), or low-poly engine proxies (Matrix-Game 4.0). AnyGameRenderer instead accepts whatever a crude game looks like, including gray blockouts, colored primitives, semantic-colored maps, wireframes, and coarse clay figures, at different granularities and with one checkpoint.

To achieve these properties, we build paired (crude video, styled video) data from three sources. (i) GEAR's multi-stream rendering data covers 38 agent-made games, helping the model generalize across genres and input types. (ii) Two-stream rendering in Unreal Engine uses scenes scripted with same-color groups that change count mid-clip, entity sizes from 20 to 600 px, and frame-aligned appearance and disappearance, teaching the model to keep entity correspondence. (iii) Real gameplay recordings, converted into crude inputs by 3D-box detection, help the model render realistic scenes. Together they yield 10,330 training clips.

Existing renderers are evaluated at the frame level, with edge agreement, point tracks, or pairwise judges, which cannot tell whether each entity is rendered correctly. We therefore design an entity-level benchmark of 59 held-out cases with 753 entities, each annotated with ground-truth boxes, and evaluate it with four metrics. *Temporal match* measures whether each entity appears and disappears on time; *position match* measures whether it stays at the right place; *unrendered rate* measures how often crude input blocks are left unrendered; and *hallucination rate* measures how often the output contains entities that do not exist in the input. Compared with the zero-shot base, AnyGameRenderer raises temporal match from 50.9% to 62.9% and position match from 57.1% to 61.9%, and lowers the unrendered rate from 4.1% to 2.9% and the hallucination rate from 52.1% to 46.0%.

Contributions.

- We propose a video-to-video generative renderer for games. A single checkpoint handles a wide range of genres and input types, renders them in diverse prompted styles, and preserves entity correspondence with the input.
- We build a paired dataset with three sources: multi-stream rendering data from GEAR, two-stream rendering data from Unreal Engine with scenarios designed for correspondence, and detection-based reconstruction from real gameplay recordings. Together they yield 10,330 training clips.
- We propose an entity-level benchmark and metrics for entity correspondence. The metrics measure whether each entity appears on time, stays in place, and is neither unrendered nor hallucinated. On this benchmark, AnyGameRenderer improves temporal match from 50.9% to 62.9% and reduces missed count changes from 34.7% to 21.3% over the zero-shot base.

2 Related Work

2.1 AI-driven game creation

Early work uses large language models to generate game content such as levels and rules. MarioGPT [34] generates Super Mario Bros. levels from text, Word2World [28] turns stories into playable 2D tile worlds, and LLMGG [13] generates game rules and levels in the Video Game Description Language. Multi-agent

frameworks such as GameGPT [6] split game development into roles that collaborate on a full project. More recently, coding agents build complete games end-to-end. Recursive Game Creator [7] iterates designer, builder, player and reviewer roles over HTML and Godot projects; OpenGame [17] is an open-source agentic framework for web games across genres such as action and tower defense; and GEAR [1] turns a short prompt into a playable game in minutes. While these systems produce working game logic, their visuals are usually limited to simple shapes and placeholder assets, far from the art quality of commercial games.

2.2 Interactive world models

Another line of work replaces the game engine altogether: an autoregressive video model, conditioned on player actions, generates the next frames. Examples include Genie [4, 30, 2], GameNGen [36], Oasis [9], MineWorld [11], GameGen-X [5], GameFactory [41], Hunyuan-GameCraft [18, 35], the Matrix-Game line [12, 37, 10], and Yume [25, 24]. These models are strong simulators but hard to control in the ways a game needs: exact entity counts, scripted events, and rules that hold for minutes. Long-horizon memory [39, 40, 38, 22] reduces drift but does not make the world programmable.

2.3 Generative rendering for games

A growing body of work keeps the engine in charge of game state and uses a generative model only to render pixels. Neural forward rendering from G-buffers [19, 3, 42, 23] and earlier generative world renderers [16, 20] follow this split, but require engine-exact inputs. Several concurrent systems extend it to *crude* inputs that a game or an agent can produce cheaply. CWM [8] lets a coding agent maintain world state and renders a 12-class proxy video with a MiniMax-H3 LoRA. PWM [15] converts real AAA recordings into state-augmented 3D oriented boxes and renders them with a structured spatial ControlNet on a frozen backbone. Matrix-Game 4.0 [32] renders low-poly engine proxies in real time with a distilled causal DiT. StreamRender-H3 [29] streams a 13-class semantic map into video. SolarRenderer [33] trains a control LoRA on a frozen MiniMax-H3 with six types of structural video, passes them without type labels, and generalizes to structure types unseen in training; it also renders in four steps and supports streaming.

AnyGameRenderer differs from these systems in three ways. First, they are demonstrated mainly on realistic third-person, driving, or a few self-built 3D scenes; our method generalizes across a wide range of genres, including tower defense, auto-battlers, top-down survivors, turn-based battles, kart racing, crowd runners, arena survival, co-op games, and shooters. Second, many of them rely on a fixed intermediate representation, while we take plain video. Third, none of them measures entity-level correspondence. Frame-level metrics barely change when a small entity is missing or a group arrives late, and edge agreement even rewards leaving crude blocks unrendered. We make correspondence, including the count, timing and position of every entity, the main target of both training data and evaluation.

3 Method

AnyGameRenderer treats game rendering as reference-conditioned video editing: given a crude gameplay recording (a plain reference video) and a caption saying what each crude entity stands for, it generates the same gameplay in a target style (Figure 2). We design a caption schema to enforce entity correspondence (Section 3.2).

3.1 Task setting

Task. Let $V_{\text{in}} \in \mathbb{R}^{T\times H\times W\times 3}$ be a crude gameplay clip and c be a text prompt describing the target style and what each crude entity represents. AnyGameRenderer produces V_{out} of the same length and resolution such that (i) camera, layout, and timing match V_{in} frame by frame; (ii) every crude entity becomes its finished counterpart at the same position, apparent size, and heading, appearing and disappearing on the same frames; and (iii) appearance, including materials, lighting, character design, and environment, follows c and an optional anchor image I_{a}.

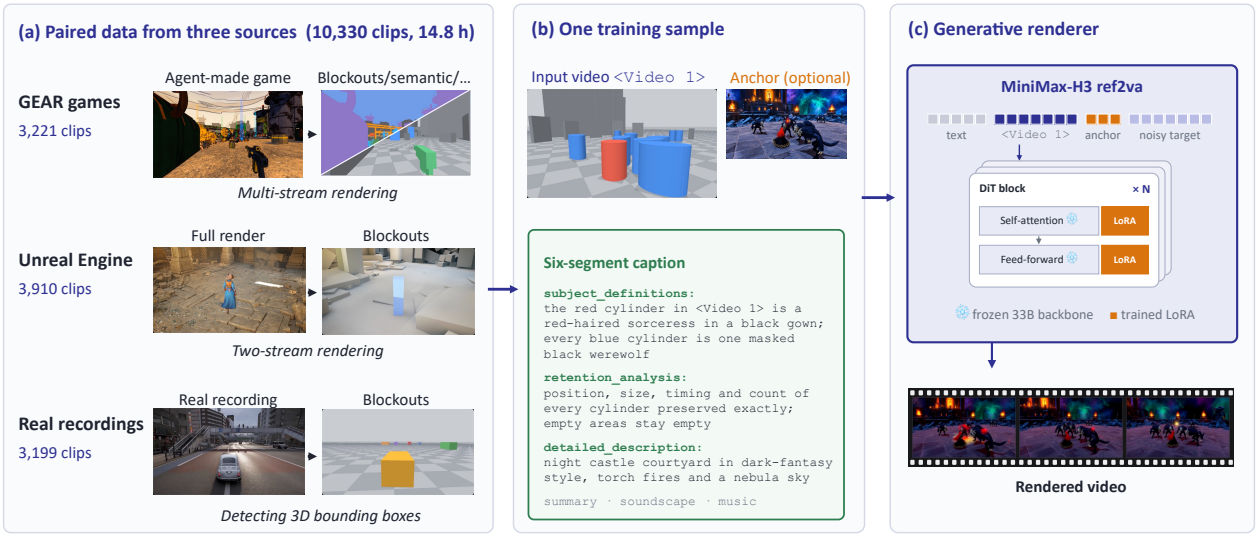


Figure 2 Overview of AnyGameRenderer. (a) Paired data from three sources. GEAR games use multi-stream rendering, which pairs each game render with several aligned crude streams such as blockouts and semantic maps. Unreal Engine scenes use two-stream rendering, which pairs a full render with blockouts. Real gameplay recordings are converted into blockouts by detecting 3D bounding boxes. (b) One training sample: the crude clip is the input video <Video 1>, the target's first frame serves as an optional anchor, and a six-segment caption defines which real entity each crude shape represents and what must be preserved. (c) Text, <Video 1>, anchor, and noisy target tokens form one sequence for the frozen MiniMax-H3 DiT. Only the LoRA adapters on self-attention and feed-forward layers are trained, and the model renders the input gameplay in the prompted style.

Base model. We build on the ref2va mode of MiniMax-H3 [27], which edits a reference video according to a text prompt. We pass the crude clip as <Video 1> and describe the target rendering in the prompt, so any crude input works. The model runs at 24 fps up to 768p. We disable the audio loss in training and discard audio at inference.

Conditioning. Each sample has three conditions: 1) the crude clip as reference video <Video 1>; 2) a six-segment structured caption (Section 3.2); and 3) an optional anchor image. During training, the anchor is the target's first frame; at inference, it serves as a style reference or is omitted. The crude clip fixes the structure, while the caption and anchor set the style. To render the same clip in a new style, we only change the prompt or the anchor.

Trainable parameters. We train a LoRA [14] of rank 128 on all transformer blocks with the standard flow-matching objective [21], keeping the base weights frozen. All reported results use 20-step Euler sampling on base+LoRA.

3.2 Caption schema: the correspondence contract

We keep the base model's six-segment format (`subject_definitions`, `summary`, `retention_analysis`, `detailed_description`, `overall_soundscape`, `non_diegetic_music`) and follow four rules, which we call the correspondence contract.

Subjects are entities; crude shapes are their representation. Each subject is defined as the real entity, and the crude input is named as its representation, e.g., "*every blue ellipsoid in* <Video 1> *is one raccoon-like orchard raider, and the whole blue group is a group of identical ones.*" The color-to-identity mapping is set per clip, groups are described at class level with explicit statements that they exist.

Retention analysis fixes what follows the input. This segment declares position, apparent size, timing, and head count as preserved exactly, and leaves appearance to the prompt. It also describes empty space positively: "areas empty in <Video 1> stay empty; no character is added that has no primitive in <Video 1>."

Positive assertions only. The base model has no negative prompt, and negative phrasing tends to produce what it names ("no HUD" reliably produces a HUD). All captions are therefore written as positive statements.

Programmatic captions with deliberate noise. Captions are generated automatically from engine metadata and detection results, not written by hand. Each caption lists every character that may appear in the game, even if some never show up in the clip. We keep this noise on purpose: it teaches the model to decide what exists from the video, not from the text. As a result, the model rarely adds characters that are named in the caption but missing from the input.

3.3 Training recipe

Unified format. All sources are resampled to 124-frame clips at 24 fps (5.17 s) and 1344×768, with the target's first frame as the anchor and a silent 32 kHz audio track. Latents and text embeddings are cached once per source.

Optimization. We train a rank-128 LoRA on all blocks with a global batch size of 16. The learning rate warms up over 100 steps and then follows a cosine decay, with a peak between 2×10^{-5} and 2.8×10^{-5}; weight decay is 0.01. We use bf16 precision, gradient checkpointing, and block swapping, while FlashAttention-3 [31] speeds up each step by about 30%. Training runs on 16 NVIDIA H200 GPUs (141 GB). Each step takes about 129 s, and three epochs over all 10,330 training clips take about 48 hours.

Deployment. We use a LoRA multiplier of 1.0 for in-domain inputs and styles. For out-of-domain text-only styles (no anchor image, and a style unseen in training), a multiplier of 0.7 recovers the zero-shot visual quality while keeping the correspondence gains. We recommend it as the default for open-ended styling.

4 Data

We build (crude video, styled video) pairs from three sources: multi-stream rendering data from GEAR covers different genres and input types; two-stream rendering data from Unreal Engine targets entity correspondence; and real gameplay recordings add realistic layouts, cameras, and motion. In every pair, the crude and styled videos share the same camera and layout and are frame-aligned by construction.

4.1 GEAR games

GEAR [1] is a game-creation system that builds small games end-to-end from a prompt and can export different render passes from a running game. Our multi-stream rendering data uses 38 agent-made games, covering shooters, survivors, tower defense, kart racing, arena, and co-op designs. This source contributes **3,221** clips. Each clip has five aligned passes: the target render; a primitive pass, where the scene is a gray blockout and characters are simple geometric shapes; a coarse pass, with a more detailed blockout and low-poly clay figures; a 12-class semantic map; and depth. Each clip also comes with an anchor image, a caption, and 3D boxes. We train on the pair of primitive and target.

4.2 Unreal Engine

Each Unreal Engine scene uses two-stream rendering: it is rendered from the same simulation and camera as a gray blockout with colored blocks for characters and as a full render. The protagonist is always blue, and other characters use a six-color set.

Rather than collecting generic gameplay, we script scenes around the zero-shot failures (see examples in Figure 3) to ensure entity correspondence:

- **Count changes in same-color groups.** Groups of 2 to 20 characters walk in, spawn, walk out, or despawn during the clip, in dense clusters, columns, or scattered formations. This targets crowds left as unrendered blocks and entities that appear late or never.

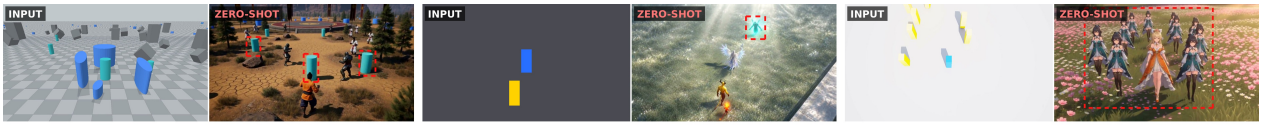


Figure 3 Three failures of the zero-shot base model. Red dashed boxes mark the errors. *Left:* input primitives are left unrendered; the cyan cylinders stay in the output instead of becoming characters. *Middle:* the input has only two blocks, but the model adds an extra creature that does not exist in the input. *Right:* the input is a top-down view, but the model renders a frontal group shot.

Table 1 Training data mixture. All clips are 124 frames at 24 fps and 1344×768, with captions generated from each source's metadata.

Source	Crude input	Clips	Share
GEAR, 38 games	primitive / coarse / semantic passes	3,221	31%
Unreal Engine	gray blockout with colored character blocks	3,910	38%
Real recordings	detected 3D boxes as solid / wireframe / cylinder / ellipsoid proxies	3,199	31%
Total		**10,330**	≈14.8 h

- **Characters of different colors entering and leaving.** This targets wrong entry and exit timing when several character types share a scene.
- **Entity sizes from 20 to 600 pixels**, including boss-sized characters and characters moving toward or away from the camera. This targets small, distant entities that dissolve into the background.
- **Mixed camera views**: 35% follow, 30% top-down, 20% isometric, and 15% side. This keeps the model from relying on a single viewpoint.

We keep clips where the protagonist is visible in at least 95% of frames. Each clip carries metadata on the scene, characters, colors, and events, from which its caption is generated. This source contributes **3,910** clips.

4.3 Real gameplay recordings

Following the reconstruction approach in PWM [15], we convert real gameplay recordings of combat, driving, and pedestrian scenes into crude inputs. We detect 3D bounding boxes in each frame and render them over a checkerboard ground as proxies of varied shapes: solid boxes, wireframe boxes, cylinders, and upright ellipsoids, with about 800 clips per shape. The original recording is the target, and captions come from the detection labels plus one sentence describing the environment. This source adds real-world scenes, with realistic camera work and vehicle and pedestrian motion. It contributes **3,199** clips.

The three sources are mixed at roughly equal weight (Table 1), giving 10,330 clips of 124 frames, about 14.8 hours of paired video.

5 Experiments

We first describe the evaluation protocol (Section 5.1), then evaluate entity correspondence (Section 5.2) and generalization across genres (Section 5.3).

5.1 Evaluation protocol

Benchmark. The benchmark contains 59 cases with 753 annotated entities: 11 held-out two-stream rendering cases from Unreal Engine (gray blockouts), 14 real recordings with detected solid boxes, and 34 multi-stream rendering cases from GEAR across 6 held-out games. Each case uses a source clip excluded from training, rendered in a style or with characters not seen in training. All models are run with the same seed, 20 sampling steps, and the same captions.

Table 2 Entity-sync results (%) on the held-out benchmark: four main metrics (left) and three secondary statistics (right). *GT floor* (gray) runs the ground-truth RGB video through the same detection and matching pipeline.

Split	Model	Temporal match ↑	Position match ↑	Unrendered rate ↓	Halluc. rate ↓	On-time appear. ↑	Never appears ↓	Missed count changes ↓
All (59)	Zero-shot	50.9	57.1	4.1	52.1	55.9	26.0	34.7
	Ours	**62.9**	**61.9**	**2.9**	**46.0**	**67.6**	**20.2**	**21.3**
	GT floor	80.9	79.3	0.1	34.1	93.6	0.7	0.0
Multi-stream rendering (34)	Zero-shot	47.6	56.4	4.5	55.4	50.9	28.4	40.7
	Ours	**60.7**	**60.6**	**3.2**	**51.2**	**64.0**	**23.5**	**23.7**
	GT floor	79.3	76.8	0.1	35.7	91.5	0.9	0.0
Two-stream rendering (11)	Zero-shot	80.2	68.1	4.9	38.9	82.3	15.2	0.0
	Ours	**96.6**	**80.6**	**1.4**	**19.4**	**94.9**	**0.0**	**0.0**
	GT floor	98.5	86.7	0.3	16.6	100.0	0.0	0.0
Real game (14)	Zero-shot	46.5	52.4	**2.6**	52.1	61.3	22.6	20.0
	Ours	**51.1**	**53.2**	2.9	**45.4**	**66.1**	**18.5**	**20.0**
	GT floor	76.7	85.5	0.0	37.4	99.2	0.0	0.0

Entity-sync metrics. Input entities are the ground-truth 2D boxes shipped with each crude video; entities that exist for fewer than 12 frames are ignored. In the generated video, we locate characters with SAM3 open-vocabulary segmentation, prompted with the caption's subject phrases plus generic person and vehicle terms. We remove shadow and reflection detections and link the remaining detections into tracks. In each frame, input boxes and detections are matched with the Hungarian method, and a pair is accepted only if IoU ≥ 0.3 or the centroid offset is within half the input box size. We report four main metrics, averaged over entities:

- **Temporal match**: the IoU between the frames in which an entity exists in the input and the frames in which its track is in place. Late or missing appearance, lingering after despawn, and identity swaps all lower it.
- **Position match**: $1 - d/(0.5\,s)$ on matched frames, where d is the centroid offset and s is the input box size.
- **Unrendered rate**: the fraction of entity-frames in which the input block is still visible as a flat color patch.
- **Hallucination rate**: the fraction of detected entity-frames that belong to tracks matching no input entity.

We also report three secondary statistics:

- **On-time appearance**: an entity is recognized at the first frame t where it is matched in both t and $t{+}1$, giving an onset delay $\Delta_{\text{on}} = t - t_{\text{in}}$, where t_{in} is its first frame in the input. An entity is on time if $|\Delta_{\text{on}}| \leq 4$ frames ($\approx$0.17 s).
- **Never appears**: the fraction of entities that are never recognized.
- **Missed count changes**: a count change is a rise of at least two entities in the input within six frames, $n_{\text{in}}[t{+}3] - n_{\text{in}}[t{-}3] \geq 2$, with target count $n_{\text{in}}[t{+}8]$. It is reproduced if, within the next 60 frames, the detected count stays at or above the target minus one for three consecutive frames. We report the fraction of the 75 count changes in the benchmark that are missed.

The *GT floor* in Table 2 runs the ground-truth RGB video through the same pipeline and marks the best score the pipeline can measure.



Figure 4 Zero-shot MiniMax-H3 versus AnyGameRenderer on held-out cases, with the same seed, steps, and caption. The zero-shot model leaves crude shapes unrendered (rows 1–4), drops the input viewpoint for out-of-domain styles (rows 5–6: a top-down blockout becomes a frontal anime scene or a first-person voxel scene), and adds caption-named characters although the input has only two blocks (row 7). AnyGameRenderer fixes all three. For rows 5–6, the "GT" column is the original in-domain render and serves only as a layout reference.

5.2 Entity correspondence

Quantitative results. Table 2 reports results on all 59 cases and on each source. Over all cases, our method improves all four main metrics: temporal match rises from 50.9% to 62.9%, position match from 57.1% to 61.9%, the unrendered rate falls from 4.1% to 2.9%, and the hallucination rate falls from 52.1% to 46.0%. The secondary statistics show the same trend. On-time appearance rises from 55.9% to 67.6%, entities that never appear fall from 26.0% to 20.2%, and missed count changes fall from 34.7% to 21.3%.

Gains differ by source. On GEAR's multi-stream rendering cases, temporal match rises by 13 points and missed count changes nearly halve. On Unreal Engine's two-stream rendering cases, temporal match reaches 96.6%, close to the GT floor of 98.5%, and no entity is missed. Real recordings improve the least: temporal match rises from 46.5% to 51.1%, the unrendered rate is similar, and missed count changes stay at 20.0%.

Qualitative comparison. Figure 4 shows seven held-out cases. The zero-shot model works well on sparse scenes with clean captions, but on game footage it fails in three ways. First, it leaves crude shapes in the output: blue cubes in an orchard, cyan cylinders in an arena, and a blue cylinder in the grass where an elk should be. Second, it drops the input viewpoint when the style is far from typical game footage, turning a top-down blockout into a frontal anime group shot or a first-person Minecraft-like scene. Third, it adds characters named in the caption that have no block in the input. AnyGameRenderer fixes all three in these cases while keeping the prompted style.

5.3 Generalization across genres

A single checkpoint handles all five genres in our benchmark: first-person shooters, top-down, third-person, crowd, and vehicle games (Figure 5). AnyGameRenderer improves temporal match over the zero-shot model in every genre, with the largest gains on crowd and third-person games. It also lowers the unrendered rate in every genre except crowd games, where both models are already near zero, and improves position match most on vehicle games. The hallucination rate falls on crowd games but rises slightly on top-down and third-person games. Top-down games remain the hardest, with low temporal match for both models.

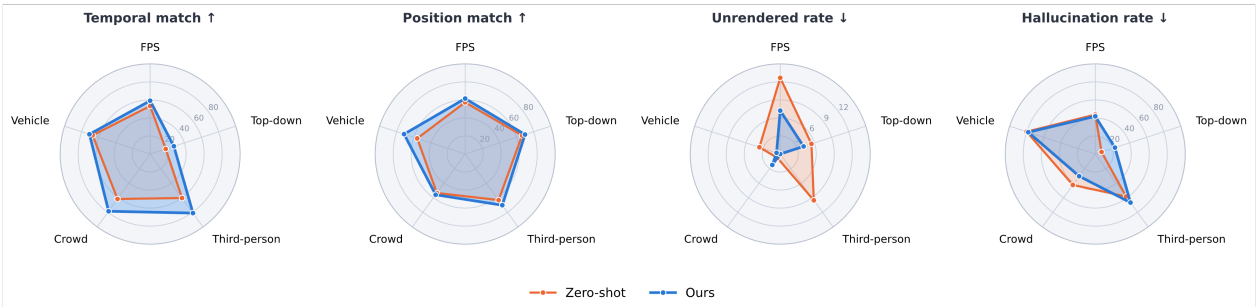


Figure 5 Entity-sync metrics by genre. Zero-shot MiniMax-H3 (orange) versus AnyGameRenderer (blue) on the held-out benchmark, grouped into first-person shooter, top-down, third-person, crowd, and vehicle games.

6 Conclusion

We presented AnyGameRenderer, a video-to-video generative renderer that turns the programmer art of agent-made games into high-quality visuals. We train a single LoRA on MiniMax-H3, using paired data from GEAR's multi-stream rendering, Unreal Engine's two-stream rendering, and real gameplay recordings. The resulting checkpoint handles a wide range of genres. It takes plain video as input, from gray blockouts and colored primitives to semantic maps and wireframes. It also keeps entities in sync with the input. On a held-out benchmark, temporal match rises from 50.9% to 62.9%, and missed count changes fall from 34.7% to 21.3%. As coding agents build more and more games, we believe a general renderer that keeps every entity in place will help these games look as good as they play.

References

[1] Alaya Lab. Gear. https://gearzero.ai/, 2026. Accessed October 20, 2026.

[2] P. Ball, J. Bauer, et al. Genie 3: A new frontier for world models. https://deepmind.google/discover/blog/genie-3-a-new-frontier-for-world-models/, 2025.

[3] O. Beisswenger, J.-N. Dihlmann, and H. Lensch. Framediffuser: G-buffer-conditioned diffusion for neural forward frame rendering. *arXiv preprint arXiv:2512.16670*, 2025.

[4] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.

[5] H. Che, X. He, Q. Liu, C. Jin, and H. Chen. Gamegen-x: Interactive open-world game video generation. In *International Conference on Learning Representations (ICLR)*, pages 37546–37593, 2025.

[6] D. Chen, H. Zhang, H. Wang, Y. Huo, Y. Li, and J. Wang. Gamegpt: Multi-agent collaborative framework for game development. *arXiv preprint arXiv:2310.08067*, 2023.

[7] J. Chen, H. Wu, M. Jia, and X. Liu. Recursive game creator: An agentic product-level experience-oriented game harness. *arXiv preprint arXiv:2610.08621*, 2026.

[8] Y. Chen, G. Lin, and C. Zhang. Code world model: Coding agent as world brain. *arXiv preprint arXiv:2608.25927*, 2026.

[9] E. Decart, Q. McIntyre, S. Campbell, X. Chen, and R. Wachen. Oasis: A universe in a transformer. https://oasis-model.github.io, 2024.

[10] R. Feng, H. Zhang, Z. Shu, Z. Yang, L. Tang, Z. Wang, A. Zheng, J. Xiao, Z. Liu, R. Chu, et al. The matrix: Infinite-horizon world generation with real-time moving control. *Advances in Neural Information Processing Systems (NeurIPS)*, 38:87318–87344, 2026.

[11] J. Guo, Y. Ye, T. He, H. Wu, Y. Jiang, T. Pearce, and J. Bian. Mineworld: a real-time and open-source interactive world model on minecraft. *arXiv preprint arXiv:2504.08388*, 2025.

[12] X. He, C. Peng, Z. Liu, B. Wang, Y. Zhang, Q. Cui, F. Kang, B. Jiang, M. An, Y. Ren, et al. Matrix-game 2.0: An open-source real-time and streaming interactive world model. *arXiv preprint arXiv:2508.13009*, 2025.

[13] C. Hu, Y. Zhao, and J. Liu. Game generation via large language models. In *2024 IEEE conference on games (CoG)*, pages 1–4. IEEE, 2024.

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[15] Z.-H. Huang, G. Lin, J. Lin, Y.-C. Huang, R. Yu, M. Niu, S. Yang, Y.-L. Liu, Y.-Y. Chuang, K. Zhang, and Z. Wang. Programmable world model. *arXiv preprint arXiv:2609.10540*, 2026.

[16] Z.-H. Huang, Z. Wang, J. Tan, R. Yu, Y. Zhang, B. Zheng, Y.-L. Liu, Y.-Y. Chuang, and K. Zhang. Generative world renderer. *arXiv preprint arXiv:2604.02329*, 2026.

[17] Y. Jiang, J. Hu, Q. Xiao, Y. Zheng, R. Ma, K. Feng, J. Han, T. Peng, K. Fan, M. Zhang, et al. Opengame: Open agentic coding for games. *arXiv preprint arXiv:2604.18394*, 2026.

[18] J. Li, J. Tang, Z. Xu, L. Wu, Y. Zhou, S. Shao, T. Yu, Z. Cao, and Q. Lu. Hunyuan-gamecraft: High-dynamic interactive game video generation with hybrid history condition. *arXiv preprint arXiv:2506.17201*, 2025.

[19] R. Liang, Z. Gojcic, H. Ling, J. Munkberg, J. Hasselgren, C.-H. Lin, J. Gao, A. Keller, N. Vijaykumar, S. Fidler, et al. Diffusion renderer: Neural inverse and forward rendering with video diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 26069–26080, 2025.

[20] G. Lin, Z.-H. Huang, S. Yang, M.-H. Yang, K. Zhang, and Z. Wang. Generative world renderer at the speed of play. *arXiv preprint arXiv:2607.18703*, 2026.

[21] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[22] Z. Liu, X. Deng, S. Chen, A. Wang, Q. Guo, M. Han, Z. Xue, M. Chen, P. Luo, and L. Yang. Worldweaver: Generating long-horizon video worlds via rich perception. *Advances in Neural Information Processing Systems (NeurIPS)*, 38:46268–46293, 2026.

[23] J. Lu, R. Xie, X. Jin, Z. Wu, Q. Ye, T. Xie, H. Bao, R. W. Huo, et al. Germ: A generative rendering model from physically realistic to photorealistic. *arXiv preprint arXiv:2604.09304*, 2026.

[24] X. Mao, Z. Li, C. Li, X. Xu, K. Ying, and K. Zhang. Yume1.5: A text-controlled interactive world generation model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7752–7761, 2026.

[25] X. Mao, S. Lin, Z. Li, C. Li, W. Peng, T. He, J. Pang, M. Chi, Y. Qiao, and K. Zhang. Yume: An interactive world generation model. *arXiv preprint arXiv:2507.17744*, 2025.

[26] Z. Meng, Z. Li, C. Li, Q. Li, and K. Zhang. Marionette: Predicting world states, rendering geometry, painting appearance. *arXiv preprint arXiv:2608.14530*, 2026.

[27] MiniMax. Minimax-h3: An open omni-modal video generation model. https://huggingface.co/MiniMaxAI/MiniMax-H3, 2026. Model weights and code released August 2026 under the MiniMax H3 Community License.

[28] M. U. Nasir, S. James, and J. Togelius. Word2world: Generating stories and worlds through large language models. *arXiv preprint arXiv:2405.06686*, 2024.

[29] NVIDIA. Streamrender-h3: Streaming semantic-map-to-video rendering with MiniMax-H3. Technical report, 2026. Concurrent work, released October 2026.

[30] J. Parker-Holder, P. Ball, J. Bruce, V. Dasagi, K. Holsheimer, C. Kaplanis, A. Moufarek, G. Scully, J. Shar, J. Shi, et al. Genie 2: A large-scale foundation world model. https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model, 2024.

[31] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.

[32] Skywork AI. Matrix-game 4.0: Code builds the game, AI renders the world. Technical report, 2026. Concurrent work, released October 2026.

[33] SolarRenderer Team. Solarrenderer: Any structure in, any world out. Technical report, 2026. Concurrent work, released October 2026.

[34] S. Sudhakaran, M. González-Duque, M. Freiberger, C. Glanois, E. Najarro, and S. Risi. Mariogpt: Open-ended text2level generation through large language models. *Advances in Neural Information Processing Systems*, 36:54213–54227, 2023.

[35] J. Tang, J. Liu, J. Li, L. Wu, H. Yang, P. Zhao, S. Gong, X. Yuan, S. Shao, L. Zhang, et al. Hunyuan-gamecraft-2: Instruction-following interactive game world model. *arXiv preprint arXiv:2511.23429*, 2025.

[36] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations (ICLR)*, pages 73754–73776, 2025.

[37] Z. Wang, Z. Liu, J. Li, K. Huang, B. Xu, F. Kang, M. An, P. Wang, B. Jiang, Y. Wei, et al. Matrix-game 3.0: Real-time and streaming interactive world model with long-horizon memory. *arXiv preprint arXiv:2604.08995*, 2026.

[38] R. Wu, X. He, M. Cheng, T. Yang, Y. Zhang, Z. Kang, X. Cai, X. Wei, C. Guo, C. Li, et al. Infinite-world: Scaling interactive world models to 1000-frame horizons via pose-free hierarchical memory. *arXiv preprint arXiv:2602.02393*, 2026.

[39] Z. Xiao, Y. Lan, Y. Zhou, W. Ouyang, S. Yang, Y. Zeng, and X. Pan. Worldmem: Long-term consistent world simulation with memory. *Advances in Neural Information Processing Systems (NeurIPS)*, 38:49632–49652, 2026.

[40] J. Yu, J. Bai, Y. Qin, Q. Liu, X. Wang, P. Wan, D. Zhang, and X. Liu. Context as memory: Scene-consistent interactive long video generation with memory retrieval. In *SIGGRAPH Asia 2025 Conference Papers*, pages 1–11, 2025.

[41] J. Yu, Y. Qin, X. Wang, P. Wan, D. Zhang, and X. Liu. Gamefactory: Creating new games with generative interactive videos. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11590–11599, 2025.

[42] S. Zhang, R. Liu, C. Schroers, and Y. Zhang. Renderflow: Single-step neural rendering via flow matching. *arXiv preprint arXiv:2601.06928*, 2026.